CrackTheHash – TryHackMe

Dificuldade: fácil

20/04/2026

Sumário

1. Visão Geral 2
2. Reconhecimento e Exploração nível 1 2
2.1 Primeira Hash 2
2.2 Segunda Hash 3
2.3 Terceira Hash 3
2.4 Quarta Hash 5
2.5 Quinta Hash 5
3. Reconhecimento e Exploração nível 2 7
3.1 Primera Hash 7
3.2 Segunda Hash 8
3.3 Terceira Hash 8
3.4 Quarta Hash 9
4. Atividade bônus 10
5. Conclusão 12
6. Referencias 12

1. Visão Geral

Para cada hash encontrada, o primeiro passo consistiu na identificação do seu tipo. Para isso, foram utilizadas as ferramentas Hash Analyzer e HashID, além da consulta a tabelas de referência, com o objetivo de determinar o algoritmo criptográfico correspondente.

Após essa etapa, foi empregada a ferramenta Hashcat para a realização do processo de quebra, utilizando o modo adequado conforme o algoritmo identificado.

Adicionalmente, utilizou-se o comando `hashcat -hh` para consultar os modos disponíveis e compreender a correspondência entre algoritmos e identificadores suportados pela ferramenta.

2. Reconhecimento e Exploração nível 1

2.1 Primeira Hash

`48bb6e862e54f2a795ffc4e541caed4d`

Hash:	48bb6e862e54f2a795ffc4e541caed4d
Hash type:	MD5 or MD4
Bit length:	128
Character length:	32
Character type:	hexidecimal

Figura 1 – HashAnalyzer da Primeira Hash

Ao analisar a primeira hash utilizando o HashAnalyzer, foi identificado que ela poderia pertencer aos algoritmos **MD5 ou MD4**.

Diante dessa hipótese, foi realizado um ataque utilizando a ferramenta Hashcat, configurada com o modo correspondente ao algoritmo MD5 (modo 0), conforme o comando abaixo:

```
hashcat -a 0 -m 0 <hash> /usr/share/wordlists/rockyou.txt
```

Após a execução, consegui descobrir a senha correspondente à primeira hash.

```
48bb6e862e54f2a795ffc4e541caed4d:easy
```

Figura 2 – Primeira hash encontrada

2.2 Segunda Hash

CBFDAC6008F9CAB4083784CBD1874F76618D2A97

Hash:	CBFDAC6008F9CAB4083784CBD1874F76618D2A97
Hash type:	SHA1 (or SHA 128)
Bit length:	160
Character length:	40
Character type:	hexidecimal

Figura 3 – HashAnalyzer da Segunda Hash

Ao usar o HashAnalyzer, foi possível identificar que se tratava de uma hash do tipo **SHA-1.** Com base nessa identificação, configurei com o modo apropriado para SHA-1 (modo 100). O comando utilizado foi:

```
hashcat -a 0 -m 100 <hash> /usr/share/wordlists/rockyou.txt
```

Após a execução do ataque, foi possível identificar com sucesso a senha correspondente à segunda hash.

```
cbfdac6008f9cab4083784cbd1874f76618d2a97:password123
```

Figura 4 – Segunda hash encontrada

2.3 Terceira Hash

1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032

Hash:	1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032\
Hash type:	unknown
Bit length:	0
Character length:	65
Character type:	Unknown

Figura 5 – HashAnalyzer da Terceira Hash

Para a terceira hash, não foi possível identificá-la inicialmente por meio do HashAnalyzer. Diante disso, utilizei a ferramenta HashID para levantar possíveis tipos de hash compatíveis.

```
┌──(kali㉿kali)-[~]
└─$ hashid 1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032
Analyzing '1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032'
[+] Snefru-256
[+] SHA-256
[+] RIPEMD-256
[+] Haval-256
[+] GOST R 34.11-94
[+] GOST CryptoPro S-Box
[+] SHA3-256
[+] Skein-256
[+] Skein-512(256)
```

Figura 6– HashId da Terceira Hash

A partir do resultado obtido, foram indicadas algumas possibilidades, sendo a primeira delas o algoritmo “Snefru-256”. Com base nessa hipótese, realizei uma busca direcionada utilizando o comando `grep`, porém não foi possível obter sucesso na identificação ou quebra da hash por esse método.

```
┌──(kali㉿kali)-[~]
└─$ hashcat -hh | grep -i snefru
```

Figura 7 – Tentativa de grep

Em seguida, analisei as demais sugestões fornecidas pela ferramenta, sendo a segunda possibilidade o algoritmo SHA-256. Com essa nova hipótese, executei o ataque utilizando o Hashcat, configurando o tipo de hash correspondente.

```
hashcat -a 0 -m 1400 <hash> /usr/share/wordlists/rockyou.txt
```

Após a execução do ataque, foi possível identificar a senha associada.

```
1c8bfe8f801d79745c4631d09fff36c82aa37fc4cce4fc946683d7b336b63032:letmein
```

Figura 8– Terceira hash encontrada

2.4 Quarta Hash

$2y$12$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom

Hash:	$2y$12$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom
Hash type:	bcrypt
Bit length:	184
Character length:	60
Character type:	$2x$x$ followed by base64
Hash:	eUznr71EeNkJkUlypTsgbX1H68wsRom
Salt:	Dwt1BZj6pcyc3Dy1FWZ5ie

Figura 9– HashAnalyzer da Quarta Hash

Inicialmente, foi utilizado o ataque padrão com wordlist, por meio do seguinte comando:

```
hashcat -a 0 -m 3200 <hash> /usr/share/wordlists/rockyou.txt
```

Entretanto, após diversas tentativas, o processo demorou excessivamente devido ao tipo de hash utilizado (bcrypt), que possui alto custo computacional, observa-se isso no parâmetro “12” presente na hash que indica o custo (cost factor), que influencia diretamente no tempo necessário para o processamento.

Considerando a dica fornecida na plataforma TryHackMe, foi criada uma nova wordlist contendo apenas senhas com exatamente 4 caracteres, reduzindo significativamente o espaço de busca.

```
grep -E '^.{4}$' /usr/share/wordlists/rockyou.txt > rockyou_4.txt
```

Com essa otimização, foi possível realizar o ataque de forma mais eficiente e, finalmente, quebrar o hash, com o seguinte comando

```
hashcat -a 0 -m 3200 <hash> rockyou_4.txt
```

```
$2y$12$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom:bleh
```

Figura 9 – Quarta hash encontrada

2.5 Quinta Hash

279412f945939ba78ce0758d3fd83daa

Para identificar o tipo do hash, utilizei ferramentas como **Hash Analyzer** e **hashid**, que indicaram possíveis formatos como **MD5 ou MD4**.

Hash:	279412f945939ba78ce0758d3fd83daa
Hash type:	MD5 or MD4
Bit length:	128
Character length:	32
Character type:	hexidecimal

Figura 10 – Quinta Hash Analyzer

Inicialmente, tentei quebrar o hash como **MD5**, porém o ataque resultou em *exhausted*, ou seja, nenhuma senha foi encontrada na wordlist utilizada.

```
hashcat -a 0 -m 0 <hash> /usr/share/wordlists/rockyou.txt
```

Em seguida, testei a hipótese de ser **MD4**, porém também não houve sucesso com as listas de palavras disponíveis.

```
hashcat -a 0 -m 900 <hash> /usr/share/wordlists/rockyou.txt
```

Como o hash não foi identificado com precisão pelas ferramentas, utilizei também o **hashid**, que sugeriu possíveis formatos adicionais, porém nenhuma das opções testadas retornou resultado.

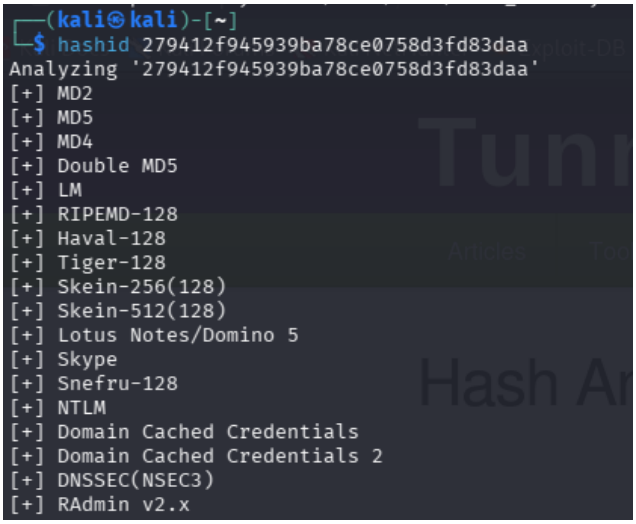

Figura 11 – Resultado Hashid

Diante disso, retornei à análise no TryHackMe, onde a dica indicava o uso de **MD4**, reforçando a hipótese inicial. No entanto, como não havia correspondência nas wordlists testadas, realizei novas tentativas utilizando outras listas e estratégias.

```
hashcat -a 0 -m 900 <hash> /usr/share/wordlists/rockyou.txt
```

```
hashcat -a 0 -m 900 <hash> /usr/share/seclists/Passwords/Common-Credentials/10k-most-common.txt
```

```
hashcat -a 0 -m 900 <hash> /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/best64.rule
```

```
hashcat -a 3 -m 900 <hash> ?d?d?d?d?d?d?d?d?d?d
```

Após diversas tentativas sem sucesso utilizando diferentes abordagens no Hashcat, optou-se pelo uso da ferramenta online CrackStation, que utiliza tabelas pré-computadas (rainbow tables). Por meio dessa abordagem, foi possível definir a senha correspondente.

Figura 12 – Resultado Crack station

3. Reconhecimento e Exploração nível 2

3.1 Primera Hash

F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85

Após a utilização do **CrackStation no nível 1**, o segundo nível apresentou maior facilidade na análise dos hashes, permitindo a obtenção de informações relevantes e contribuindo para a descoberta da resposta final.

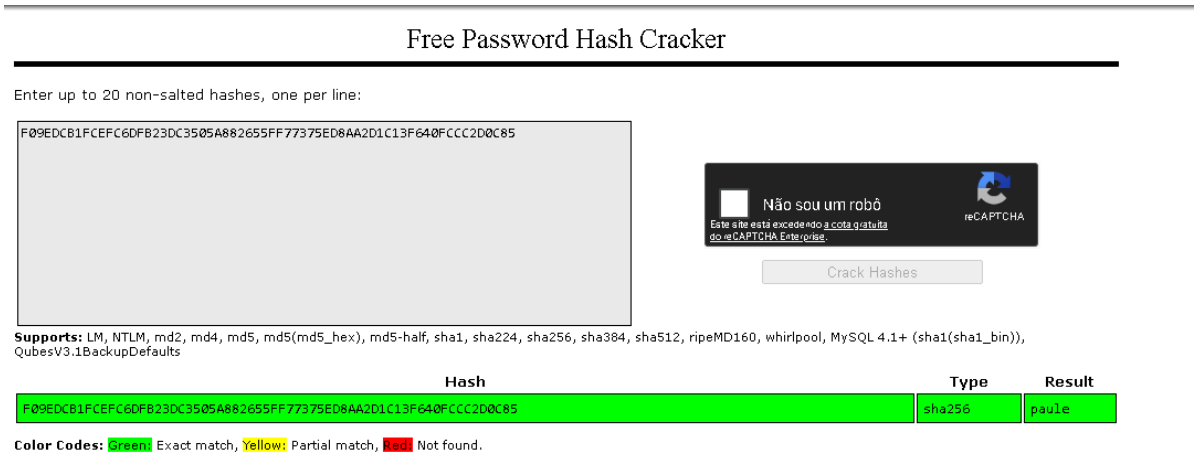

Figura 13 – Resultado primeira hash Crack station nivel 2

3.2 Segunda Hash

1DFECA0C002AE40B8619ECF94819CC1B

Utilizou-se o Crack Station:

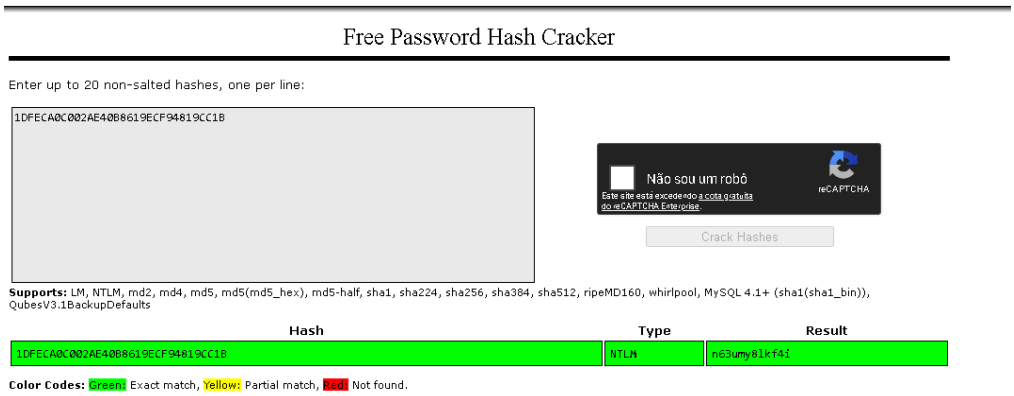

Figura 14 – Resultado Crack station segunda hash

3.3 Terceira Hash

6aReallyHardSalt$6WKUTqzq.UQQmrm0p/T7MPpMbGNnzXPMAXi4bJMl9be.cfi3/qxIf.hsGpS41BqMhSrHVXgMpdjS6xeKZAs02. Salt: aReallyHardSalt

Figura 15 – Erro ao usar o Crack station

Inicialmente não foi possível identificar o tipo do hash de forma direta, então foi realizada uma análise utilizando tabelas conhecidas de hashes. Observou-se que o formato iniciava com `$6$`, o que indica o algoritmo **SHA-512 crypt**.

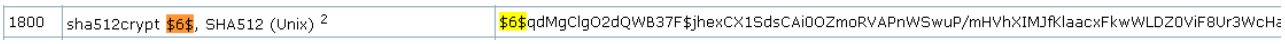

Figura 16 – Resultado Crack station segunda hash

Como tentativa de otimização, foi criada uma wordlist reduzida contendo apenas senhas com 6 caracteres, a partir da base rockyou

```
grep -E '^.{6}$' /usr/share/wordlists/rockyou.txt > rockyou_6.txt
```

Em seguida, foi realizado o ataque utilizando o Hashcat com o modo correspondente ao SHA-512 crypt:

```
hashcat -a 0 -m 1800 <hash> rockyou_6.txt
```

```
$6$aReallyHardSalt$6WKUTqzq.UQQmrm0p/T7MPpMbGNnzXPMAXi4bJMl9be.cfi3/qxIf.hsGpS41BqMhSrHVXgMpdjS6xeKZAs02.:waka99
```

Figura 17 –Senha achada

3.4 Quarta Hash

e5d8870e5bdd26602cab8dbe07a942c8669e56d6: tryhackme

Hash:	e5d8870e5bdd26602cab8dbe07a942c8669e56d6
Hash type:	SHA1 (or SHA 128)
Bit length:	160
Character length:	40
Character type:	hexidecimal

Figura 18 – Hashanylyzer da quarta hash

Incialmente, foram testadas diferentes variações do algoritmo SHA-1 com uso de salt, conforme sugerido pela ferramenta. Após várias tentativas, identificou-se que o formato correto correspondia ao **HMAC-SHA1**, o que permitiu a configuração adequada do ataque.

100	SHA1	b89eaac7e61417341b710b727768294d0e6a277b
110	sha1($pass.$salt)	2fc5a684737ce1bf7b3b239df432416e0dd07357:2014
120	sha1($salt.$pass)	cac35ec206d868b7d7cb0b55f31d9425b075082b:5363620024
130	sha1(utf16le($pass).$salt)	c57f6ac1b71f45a07dbd91a59fa47c23abcd87c2:631225
140	sha1($salt.utf16le($pass))	5db61e4cd8776c7969cfd62456da639a4c87683a:8763434884872
150	HMAC-SHA1 (key = $pass)	c898896f3f70f61bc3fb19bef222aa860e5ea717:1234
160	HMAC-SHA1 (key = $salt)	d89c92b4400b15c39e462a8caa939ab40c3aeeea:1234

Figura 19 –variações de SHA1

Utilizando então o seguinte comando no Hashcat:

```
hashcat -a 0 -m 160 <hash> /usr/share/wordlists/rockyou.txt
```

Com a definição correta do tipo de hash e a utilização da wordlist rockyou.txt, foi possível realizar o ataque com sucesso e obter a senha correspondente.

```
e5d8870e5bdd26602cab8dbe07a942c8669e56d6:tryhackme:481616481616
```

Figura 20 – Quarta hash achada

4. Atividade bônus

As hashes apresentadas nesta seção não fazem parte do laboratório *Crack The Hash* da plataforma TryHackMe, sendo incluídas apenas como exercícios complementares.

Hash: 482c811da5d5b4bc6d497ffa98491e38

Utilizou-se o Crack Station:

Hash	Type	Result
482c811da5d5b4bc6d497ffa98491e38	md5	password123

Hash: 861c4f67e887dec85292d36ab05cd7a1a7275228

Utilizou-se o Crack Station:

Hash	Type	Result
861c4f67e887dec85292d36ab05cd7a1a7275228	sha1	easy

Hash: 4149c5cc4c378444d116d65ad5ba4099

DICA: apenas letras e números (6 caracteres)

Utilizou-se a ferramenta Hash Analyzer para a identificação do tipo de hash. Inicialmente, foi realizado um ataque utilizando o Hashcat no modo MD5 (`-m 0`), porém sem sucesso na identificação da senha.

Em seguida, foi testado o modo MD4 (`-m 900`). Nessa etapa, assim como na anterior, aplicou-se uma abordagem de força bruta com máscara personalizada no Hashcat, utilizando o parâmetro `-1` para definir um conjunto de caracteres contendo letras minúsculas, maiúsculas e dígitos (`?l?u?d`). O ataque foi configurado para senhas de 6 caracteres, reduzindo o espaço de busca e direcionando a tentativa para combinações mais prováveis.

Hash:	4149c5cc4c378444d116d65ad5ba4099
Hash type:	MD5 or MD4
Bit length:	128
Character length:	32
Character type:	hexidecimal

```
hashcat -a 3 -m 900 <hash> -1 ?l?u?d ?1?1?1?1?1?1
```

```
4149c5cc4c378444d116d65ad5ba4099:0ffs3c
```

Hash: cdeb746ec095149627348b61d4140fc58b745875 Salt: satech

DICA: HMAC-SHA1

Para a análise desta hash, foi considerada a dica que indicava o uso do algoritmo **HMAC-SHA1**. Com base nessa informação, utilizou-se a ferramenta Hashcat para realizar o ataque utilizando uma wordlist.

A hash fornecida possui um valor de *salt* associado ("satech"), sendo necessário incluí-lo no formato correto (`hash:salt`) para que o processamento ocorra adequadamente.

O comando utilizado foi:

```
hashcat -a 0 -m 160 <hash>:<salt> /usr/share/wordlists/rockyou.txt
```

```
cdeb746ec095149627348b61d4140fc58b745875:satech:ovelha
```

Hash: 362fda2183b7ac73400a83f6ab2c359451e48adf6c3d46a2963ee2abdf852912

DICA: Apenas letras minúsculas e números (6 caracteres)

Para a análise desta hash, foi considerada a dica indicando que a senha é composta apenas por letras minúsculas e números, com comprimento de 6 caracteres.

Com base nessas informações, utilizou-se a ferramenta Hashcat com ataque de força bruta por máscara, definindo um conjunto de caracteres personalizado por meio do parâmetro `-1`, incluindo letras minúsculas (`?l`) e dígitos (`?d`).

O comando utilizado foi:

Hash:	362fda2183b7ac73400a83f6ab2c359451e48adf6c3d46a2963ee2abdf852912
Hash type:	SHA2-256
Bit length:	256
Character length:	64
Character type:	hexidecimal

```
hashcat -a 3 -m 1400 <hash> -1 ?l?d ?1?1?1?1?1?1
```

```
362fda2183b7ac73400a83f6ab2c359451e48adf6c3d46a2963ee2abdf852912:sawctf
```

5. Conclusão

Ao longo deste laboratório, foi possível compreender na prática o processo de identificação e quebra de hashes utilizando diferentes técnicas e ferramentas. Observou-se que a correta identificação do algoritmo é fundamental para o sucesso do ataque, bem como a escolha adequada de estratégias, como uso de wordlists, regras e ataques por máscara.

Além disso, verificou-se que algoritmos mais robustos, como bcrypt e SHA-512 crypt, apresentam maior resistência a ataques, exigindo otimizações ou informações adicionais para viabilizar sua quebra. Dessa forma, a atividade contribuiu para o entendimento dos mecanismos de segurança e das boas práticas relacionadas ao armazenamento de senhas.

6. Referencias

https://www.tunnelsup.com/hash-analyzer/
https://hashcat.net/wiki/doku.php?id=example_hashes
https://crackstation.net/